Language Modelling 5

From Random Experiments to Natural Language Processing – by Wilker Aziz. This book has not been published yet.

5.1 N-Gram LMs 23
5.2 Autoregressive LMs 25

In this Chapter we will present models that can assign probability to sequences of tokens (*e.g.*, sentences, paragraphs, or documents). This is an instance of *text generation*, rather than text analysis. In text generation, the variable of interest is *discrete and structured* (*i.e.*, finite-length sequences of discrete tokens) and its sample space is infinite (*i.e.*, the set of all sequences). For example, we want a model that can assign probability to *'Desmond has a barrow in the marketplace, Molly is the singer in a band.'* or any such sentence in English (or whatever language we choose to model), such a model is what we call a *language model* (LM). Part of the deal with a language model is to define and estimate a probability distribution that captures a 'preference' for sentences that are more likely to be accepted as valid sentences in the language. In practice an LM will prefer sentences that reproduce statistics of the observations used to estimate its parameters, whether these sentences will resemble valid sentences or not will depend on how expressive the LM is, that is, whether or not the LM can capture patterns as complex as those observed in well-formed sentences.

Let us start by talking about the **vocabulary**: the set of known discrete symbols we use to create sentences in a language is a finite set $\mathscr{W}$ that we refer to as a vocabulary. We will refer to the elements of the vocabulary as words, and we will assume our vocabulary has some finite size V. This vocabulary contains one special symbol that we shall refer to as the end-of-sequence symbol, denoted EoS. A random word W takes on symbols in the vocabulary $\mathscr{W}$ of the language. A random sentence X is a sequence of L random words: $X = \langle W_1, \ldots, W_L \rangle$, also denoted $W_{1:L}$, and, by assumption, a valid sentence always ends with the symbol EoS.

Naturally occurring sentences do not end with EoS, that is all but a modelling convenience we agree upon, so we artificially append this EoS symbol to existing sentences.

A **language model** is a probability distribution over the set $\mathscr{W}^*$, that is, the set of all finite-length sequences of symbols in $\mathscr{W}$. A sentence is drawn from the model as follows:

1. Start from an empty history $\langle\rangle$,
2. Repeat
   - Condition on the available history of already sampled words $H = w_{<i}$ and draw a word w_i with probability $P(W = w_i | H = w_{<i})$ extending the history with it.
   - If w_i is EoS terminate the algorithm and return the sentence.

This model assigns probability $P(X = w_{1:l})$ to the sentence $w_{1:l} \in \mathscr{W}^*$ given by:

$$P(X = w_{1:l}) = \prod_{i=1}^{l} P(W = w_i | H = w_{<i}) \tag{5.1}$$

Example:

$$\begin{aligned} &P_X(\langle \text{He, went, to, the, store, EOS} \rangle) \\ &= P_{W|H}(\text{He}|\langle\rangle) \\ &\quad \times P_{W|H}(\text{went}|\langle \text{He} \rangle) \\ &\quad \times P_{W|H}(\text{to}|\langle \text{He, went} \rangle) \\ &\quad \times P_{W|H}(\text{the}|\langle \text{He, went, to} \rangle) \\ &\quad \times P_{W|H}(\text{store}|\langle \text{He, went, to, the} \rangle) \\ &\quad \times P_{W|H}(\text{EOS}|\langle \text{He, went, to, the, store} \rangle) \end{aligned}$$

where we use $w_{<i}$ to denote the sequence of words that precede the ith position. This is an application of chain rule of probabilities to re-express the joint probability of the entire sequence as a product of conditional

probabilities for words given their history. Naively parameterising the chain rule would lead to difficulties, since the history of the conditional factors keeps on growing at each step.

5.1 N-Gram LMs

A classic n-gram LM is an LM that makes a strong conditional independence assumption, namely, that at each point, the next word W depends on the history H only through the most recent $n-1$ words:

$$P(X = w_{1:l}) \overset{\text{ind.}}{=} \prod_{i=1}^{l} P(W = w_i | H = w_{i-n+1:i-1}) \,. \tag{5.2}$$

Example using trigrams:

$$\begin{aligned} & P_X(\langle \text{He, went, to, the, store, EOS} \rangle) \\ & \overset{\text{ind.}}{=} P_{W|H}(\text{He}|\langle \text{BoS, BoS} \rangle) \\ & \quad \times P_{W|H}(\text{went}|\langle \text{BoS, He} \rangle) \\ & \quad \times P_{W|H}(\text{to}|\langle \text{He, went} \rangle) \\ & \quad \times P_{W|H}(\text{the}|\langle \text{went, to} \rangle) \\ & \quad \times P_{W|H}(\text{store}|\langle \text{to, the} \rangle) \\ & \quad \times P_{W|H}(\text{EOS}|\langle \text{the, store} \rangle) \end{aligned}$$

In a trigram LM, we condition on the last 2 words in the history. To make all histories have size 2, we prepend 2 occurrences of a special begin-of-sentence (BoS) symbol to the sequence (we do not assign probability to those, we simply assume they occur in the history).

Early n-gram LMs were directly parameterised using tabular Categorical cpds. That is, for each unique history $h \in \mathscr{W}^{n-1}$, we would store a Categorical distribution:

$$W|H = h \sim \text{Categorical}(\pi^{(h)}_{1:V}) \,. \tag{5.3}$$

Note that the number of unique histories grows with $\mathcal{O}(V^{n-1})$, and because for each unique history we need to store and estimate V parameters, the total model size grows in the order of $\mathcal{O}(V^n)$. Unless V is very small, this is typically very challenging for $n > 3$. Given a dataset of N observations $\mathscr{D} = \{x^{(1)}, \ldots, x^{(N)}\}$, each a sequence in $\mathscr{W}^*$, we can estimate the parameters of these LMs via MLE:

$$\pi^{(h)}_w = \frac{\text{count}_{HW}(h, w)}{\sum_{o=1}^{V} \text{count}_{HW}(h, o)} \tag{5.4}$$

where $\text{count}_{HW}(h, w)$ counts the number of joint occurrences of $(H = h, W = w)$ in $\mathscr{D}$, that is, the number of times the sequence $h \circ \langle w \rangle$ occurred in the data.

We use $a \circ b$ to indicate the concatenation of two sequences.

Unfortunately, for very sparse cpds (*e.g.*, those conditioned on longer histories), MLE is very unreliable, and even techniques like Laplace smoothing are relatively unable to make good use of the data. A classic n-gram LM (from 90s and early 2000s) had to rely on more sophisticated strategies to smooth maximum likelihood estimation (*e.g.*, Kneser-Ney smoothing, backoff smoothing, interpolation, and many other techniques we will not be discussing in this chapter).

An alternative parameterisation of a classic n-gram LM uses vector representations of words and histories for compactness and increased statistical efficiency. The idea is to parameterise Categorical distributions in context using a single neural network architecture $\boldsymbol{\pi}(\cdot;\theta)$:

$$W|H = w_{i-n+1:i-1} \sim \text{Categorical}(\boldsymbol{\pi}(w_{i-n+1:i-1};\theta)) \tag{5.5a}$$
$$\mathbf{e}_j = \text{emb}_D(w_j;\theta_{\text{in}}) \tag{5.5b}$$
$$\mathbf{u}_i = \text{concat}(\mathbf{e}_{i-n+1},\ldots,\mathbf{e}_{i-1}) \tag{5.5c}$$
$$\mathbf{s}_i = \text{ffnn}_V(\mathbf{u}_i;\theta_{\text{out}}) \tag{5.5d}$$
$$\boldsymbol{\pi}(w_{i-n+1:i-1};\theta) = \text{softmax}(\mathbf{s}_i) \tag{5.5e}$$

As it can be seen by applying *the same* NN over and over, each time with different inputs (*i.e.*, different histories), we can parameterise any cpd we need using a number of parameters that does not grow exponentially with V and n.

Exercise 5.1.1

How many parameters are needed to specify the language model above? Express it as a function of vocabulary size V, embedding dimensionality D, size of hidden layer H (assume a single hidden layer), and n-gram size.

This time, we do not have a closed-form expression for the maximum likelihood estimate of the NN parameters, but we can use gradient-based search to find a local maximum of the log-likelihood function (or a local minimum of the negative log-likelihood).

Given a single observation $X = w_{1:l}$, the log-likelihood function is

$$\mathcal{L}(\theta|w_{1:l}) = \log f_X(w_{1:l};\theta) \tag{5.6a}$$

where $f_X(x;\theta)$ is the pmf of the joint distribution over X, which factorises using the pmfs $f_{W|H}(w|h;\theta)$ of the conditional distributions for each history:

$$= \sum_{i=1}^{l} \log f_{W|H}(w_i|w_{i-n+1:i-1};\theta) \tag{5.6b}$$

which, in turn, are Categorical pmfs

$$= \sum_{i=1}^{l} \log \text{Categorical}(w_i|\boldsymbol{\pi}(w_{i-n+1:i-1};\theta)) \tag{5.6c}$$

and the probability of word w_i in context is in the coordinate of the NN output that corresponds to that word:

$$= \sum_{i=1}^{l} \log\,[\pi(w_{i-n+1:i-1};\theta))]_{w_i}. \tag{5.6d}$$

With N observations $\mathcal{D} = \{x^{(1)},\ldots,x^{(N)}\}$ we accumulate $\mathcal{L}(\theta|x^{(n)})$ for each data point.

An *embedding layer* takes a categorical input t from a vocabulary of V known categories and maps it to a D-dimensional output as follows:

$$\mathbf{e} = \mathbf{E}_t^\top \text{onehot}_V(a) = \mathbf{E}_{\text{id}(t)}$$

where $\text{onehot}_V(t)$ is a V-dimensional vector whose coordinates are all 0 except for the coordinate corresponding to t, the matrix $\mathbf{E} \in \mathbb{R}^{V\times D}$ are the trainable parameters of the embedding layer. In this chapter, we denote this more compactly as $\mathbf{e} = \text{emb}_D(t;\theta)$ where the subscript indicates the dimensionality of the output, and θ is the set of trainable parameters of the layer (*i.e.*, $\theta = \{\mathbf{E}\}$). When we use multiple such layers, each with its own parameters, we indicate that by naming their parameters differently. For example, in a given model we might use $\text{emb}(\cdot;\theta_{\text{en}})$ and $\text{emb}(\cdot;\theta_{\text{nl}})$ to denote embedding layers for English and Dutch, respectively.

A *line layer* maps an I-dimensional input $\mathbf{x}$ to an O-dimensional output $\mathbf{y}$ through a linear transformation:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

the trainable parameters are $\mathbf{W} \in \mathbb{R}^{O\times I}$ and $\mathbf{b} \in \mathbb{R}^O$. We denote this operation compactly as $\mathbf{y} = \text{linear}_O(\mathbf{x};\theta)$, where the subscript indicates the dimensionality of the output and $\theta = \{\mathbf{W},\mathbf{b}\}$.

A *single-hidden-layer feed-forward neural network* maps an I-dimensional input $\mathbf{x}$ to an O-dimensional output $\mathbf{y}$, it does so by first projecting $\mathbf{x}$ to an H-dimensional 'hidden representation' $\mathbf{h}$ and then projecting $\mathbf{h}$ to an O-dimensional output:

$$\mathbf{h} = \tanh\left(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\right)$$
$$\mathbf{y} = \mathbf{W}^{[2]}\mathbf{h} + \mathbf{b}^{[2]}$$

the projections are themselves linear with $\mathbf{W}^{[1]} \in \mathbb{R}^{H\times I}, \mathbf{b}^{[1]} \in \mathbb{R}^H$ and $\mathbf{W}^{[2]} \in \mathbb{R}^{O\times H}, \mathbf{b}^{[2]} \in \mathbb{R}^O$, but the hidden representation $\mathbf{h}$ in the middle is transformed non-linearly. Common non-linearities for hidden layers include tanh, sigmoid and ReLU. We could have presented this architecture using the linear block, for example:

$$\mathbf{h} = \tanh(\text{linear}_H(\mathbf{x};\theta_{\text{hid}}))$$
$$\mathbf{y} = \text{linear}_O(\mathbf{h};\theta_{\text{out}})$$

which makes our description compact and very similar to how these functions are composed in software packages. We denote this operation compactly as $\mathbf{y} = \text{ffnn}_O(\mathbf{x};\theta)$ where the subscript indicates the dimensionality of the output and $\theta = \{\mathbf{W}^{[1]},\mathbf{b}^{[1]},\mathbf{W}^{[2]},\mathbf{b}^{[2]}\}$. Feed-forward nets can have multiple hidden layers, each built upon a choice of linear transformation and a choice of non-linearity.

5.2 Autoregressive LMs

We will now present an evolution of the neural n-gram LM, which dispenses with the independence assumption altogether. This model factorises the probability $P(X = w_{1:l})$ of a sentence $w_{1:l}$ exactly as shown in Equation 5.1, and yet it is parameterised efficiently. The difficulty in parameterising cpds of the form $W|H = w_{<i}$, is that the history $w_{<i}$ that precedes the ith word grows whenever we progress along the sequence. Doing so with a feed-forward network would not be possible, because the feed-forward network needs a fixed number of inputs. For situations where we have a variable number of inputs, we can turn to a recurrent neural network in order to encode the *entire history* in an efficient way:

$$W|H = w_{<i} \sim \text{Categorical}(\boldsymbol{\pi}(w_{<i}; \theta)) \tag{5.7}$$

$$\mathbf{e}_j = \text{emb}_D(w_j; \theta_{\text{in}}) \tag{5.8}$$

$$\mathbf{t}_i = \text{rnnstep}_H(\mathbf{t}_{i-1}, \mathbf{e}_{i-1}; \theta_{\text{cell}}) \tag{5.9}$$

$$\mathbf{s}_i = \text{affine}_V(\mathbf{t}_i; \theta_{\text{out}}) \tag{5.10}$$

$$\boldsymbol{\pi}(w_{<i}; \theta) = \text{softmax}(\mathbf{s}_i)\ . \tag{5.11}$$

Models built upon an exact factorisation of the chain rule, as this language model, are also known as *autoregressive* models, thus it is common to talk about an RNN LM as an autoregressive LM.

Exercise 5.2.1

How many parameters are needed to specify the language model above? Express it as a function of vocabulary size V, embedding dimensionality D, and size H of hidden layer in the recurrent cell.

As before, given an observation $X = w_{1:l}$, the log-likelihood function is

$$\mathcal{L}(\theta|w_{1:l}) = \sum_{i=1}^{l} \log \text{Categorical}(w_i|\boldsymbol{\pi}(w_{<i}; \theta)) \tag{5.12a}$$

$$= \sum_{i=1}^{l} \log\, [\pi(w_{<i}; \theta))]_{w_i}\ , \tag{5.12b}$$

but this time the probability of each word in the sequence depends on every word in the complete history up until that point.

A *recurrent neural network* (RNN) maps a sequence $\mathbf{e}_1, \ldots, \mathbf{e}_L$ of L inputs, each I-dimensional, to a sequence of L outputs, each O-dimensional. The network is built upon a simple block which it uses recurrently from left to right. The general computations performed by an RNN can be abstracted as follows:

$$\text{for } i = 1, \ldots, L$$
$$\mathbf{h}_i = \text{rnnstep}_H(\mathbf{h}_{i-1}, \mathbf{e}_{i-1}; \theta_{\text{cell}})$$

where $\mathbf{h}_0$ and $\mathbf{e}_0$ are often assumed to be vectors of zeros. There are different implementations of the rnnstep function, each implementation leads to a different type of RNN. One example is the Elman step:

$$\mathbf{h}_i = \tanh(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{e}_{i-1})$$
$$\theta_{\text{cell}} = \{\mathbf{U}, \mathbf{W}\}$$
$$\mathbf{U} \in \mathbb{R}^{H\times H}, \mathbf{W} \in \mathbb{R}^{H\times I}$$

where we project the state and the input to H dimensions each and apply a nonlinearity to their sum. The important aspect of the RNN is that at each step i, the hidden state $\mathbf{h}_i$ is potentially sensitive to all inputs $\mathbf{e}_{<i}$ prior to $\mathbf{e}_i$ in the specific order they were presented. You can see this by unrolling the RNN steps, for example for $L = 4$:

$$\mathbf{h}_1 = \text{rnnstep}(\mathbf{h}_0, \mathbf{e}_0; \theta_{\text{cell}})$$
$$\mathbf{h}_2 = \text{rnnstep}(\mathbf{h}_1, \mathbf{e}_1; \theta_{\text{cell}})$$
$$\mathbf{h}_3 = \text{rnnstep}(\mathbf{h}_2, \mathbf{e}_2; \theta_{\text{cell}})$$
$$\mathbf{h}_4 = \text{rnnstep}(\mathbf{h}_3, \mathbf{e}_3; \theta_{\text{cell}})\ .$$

Crucially, the RNN reuses the same parameters θ_{cell} for each and every step of the way. This is what allows us to processes a sequence of arbitrary (but finite) length while using a fixed number of parameters, which in no way depends on L itself. The computation time to run the RNN does depend on L, because we make one call to 'rnnstep' per step of the sequence, but the number of parameters does not depend on that.

You may remember the RNN as a text encoder for text classification, there are subtle but crucial differences when we use the RNN as part of a text generator. See Figure 5.1 to better understand the difference.

Other popular step functions include the *long short-term memory* (LSTM) and the *gated recurrent unit* (GRU). Those are the two RNN cells you are more likely to interact with in a modern software package.

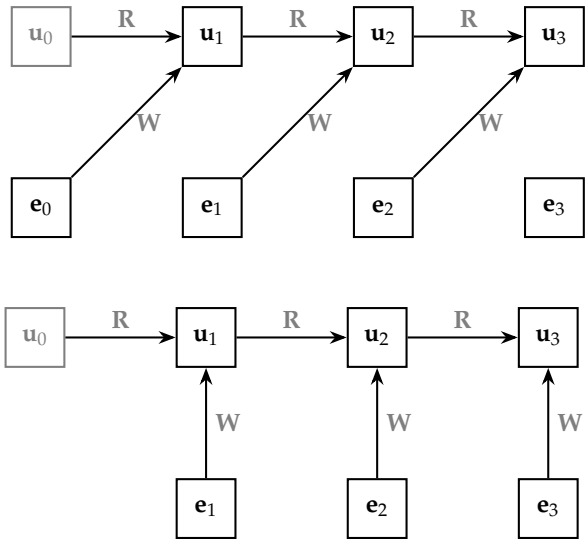

Figure 5.1: Recurrent neural network unfolded for 3 steps in either *decoding* (top) or *encoding* (bottom) mode. In gray, we depict the parameters of the RNN: the initial state $\mathbf{u}_0 \in \mathbb{R}^H$ and the matrices $\mathbf{R} \in \mathbb{R}^{H \times H}$ and $\mathbf{W} \in \mathbb{R}^{H \times I}$. At each step $i \in [l]$ of the process, the RNN uses the same two matrices ($\mathbf{W}$ and $\mathbf{R}$) to compose a recurrent state $\mathbf{u}_i$ as a function of the preceding recurrent state $\mathbf{u}_{i-1}$ and an input (this is $\mathbf{e}_{-1}$ in decoding mode or $\mathbf{e}_i$ in encoding mode). One of the simplest RNN cells uses the transformation: $\mathbf{u}_i = \tanh(\mathbf{R}\mathbf{u}_{i-1} + \mathbf{W}\mathbf{e}_{i-1})$ for decoders, or $\mathbf{u}_i = \tanh(\mathbf{R}\mathbf{u}_{i-1} + \mathbf{W}\mathbf{e}_i)$ for encoders. The RNN decoder is such that its ith state $\mathbf{u}_i$ is a function of all inputs *before* $\mathbf{e}_i$, hence not including $\mathbf{e}_i$, while the RNN encoder is such that its ith state $\mathbf{u}_i$ is a function of all inputs before $\mathbf{e}_i$ *as well as* $\mathbf{e}_i$. This subtle difference means that we can use the state $\mathbf{u}_i$ of the RNN decoder in applications where we are supposed to predict a probability distribution from which we will draw the next token w_i (which correponds to the input $\mathbf{e}_i$) or which we will use to assign probability to the next observed token w_i. If we accidentally used an RNN encoder for such an application, we would be conditioning on w_i via $\mathbf{e}_i$, which would lead to an infeasible model (since when we are generating data with this model we cannot possibly know w_i in order to predict the distribution from which we sample w_i).